Проект «Нейро-стенографист» (Hex.Team)

Хакатон Nuclear IT Hack · описание проекта и распределение ролей

Что делаем

Веб-приложение, которое принимает запись совещания и выдаёт готовый протокол: резюме, темы, решения и задачи с ответственными и сроками. Вместо часа ручной работы — несколько минут.

Как работает

• Загружаем аудио.
• Распознаём речь в текст с таймкодами (Whisper).
• Определяем, кто говорит: «Спикер 1», «Спикер 2» (диаризация).
• LLM (Kimi или Claude) собирает структурированный протокол.
• Вопросы по записи: ответ со ссылкой на таймкод.
Стек: Python, Streamlit, OpenAI-совместимый API, Git/GitHub. **Чем выделяемся:** не просто «аудио в текст», а протокол с задачами и поиск по записи.

Роли в команде

Участник	Роль	Задачи и файлы
1. _____	**Капитан / интегратор**	Репозиторий, ветки, мерджи, **.env.example**, **requirements.txt**; **pipeline.py**; ревью кода; подача формы
2. _____	**ASR + диаризация**	**asr.py**, **diarization.py**; сбор 3–4 тестовых записей (чистая, шумная, с перебиванием)
3. _____	**LLM и промпты**	**llm.py** (Kimi + запасная модель, JSON, повторы); доработка **prompts.py**; длинные записи (chunk/merge); QA по записи
4. _____	**Frontend + презентация**	**app.py** (Streamlit): загрузка, прогресс, протокол, экспорт MD/DOCX; слайды и сценарий демо

Контракты данных (фиксируем в первый час)

• **transcribe(path)** → list[{start, end, text}]
• **diarize(path)** → list[{start, end, speaker}]
• **merge(segments, speakers)** → строки «[MM:SS] Спикер N: текст»
• **make_minutes(transcript)** → dict по схеме MINUTES_SCHEMA
• Пока модуль не готов — работаем на заглушках (готовый .txt и фиктивный JSON).

Правила работы

• Каждый работает в своей ветке и коммитит после каждого рабочего шага.
• Секреты (API-ключи) только в .env, в репозиторий не попадают.
• **app.py** только интерфейс, вся логика в отдельных модулях.
• Этапы: заглушки и модули → интеграция и тест на записях → доработки → только стабильность и демо.
• Если людей меньше — объединяем роли 2 и 3, диаризацию делаем последней.